

# **Лабораторная работа № 10**

**Программирование в командном процессоре ОС UNIX. Командные  
файлы**

Жукова София Викторовна

# Содержание

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                    | <b>5</b>  |
| <b>2</b> | <b>Задание</b>                        | <b>6</b>  |
| <b>3</b> | <b>Выполнение лабораторной работы</b> | <b>7</b>  |
| <b>4</b> | <b>Выводы</b>                         | <b>14</b> |

## Список иллюстраций

|      |                                                                 |    |
|------|-----------------------------------------------------------------|----|
| 3.1  | Создание нового каталога и файла для скрипта . . . . .          | 7  |
| 3.2  | Написание первого скрипта . . . . .                             | 7  |
| 3.3  | Право на выполнение, запуск файла и проверка . . . . .          | 8  |
| 3.4  | Создание второго файла и открытие . . . . .                     | 8  |
| 3.5  | Написание второго скрипта . . . . .                             | 9  |
| 3.6  | Право на выполнение, запуск файла . . . . .                     | 9  |
| 3.7  | Создание третьего файла . . . . .                               | 10 |
| 3.8  | Написание третьего скрипта . . . . .                            | 10 |
| 3.9  | Право на выполнение, запуск файла для каталога backup . . . . . | 11 |
| 3.10 | Создание четвёртого файла . . . . .                             | 12 |
| 3.11 | Написание четвёртого скрипта . . . . .                          | 12 |
| 3.12 | Право на выполнение, запуск файла для .txt . . . . .            | 13 |

## **Список таблиц**

# 1 Цель работы

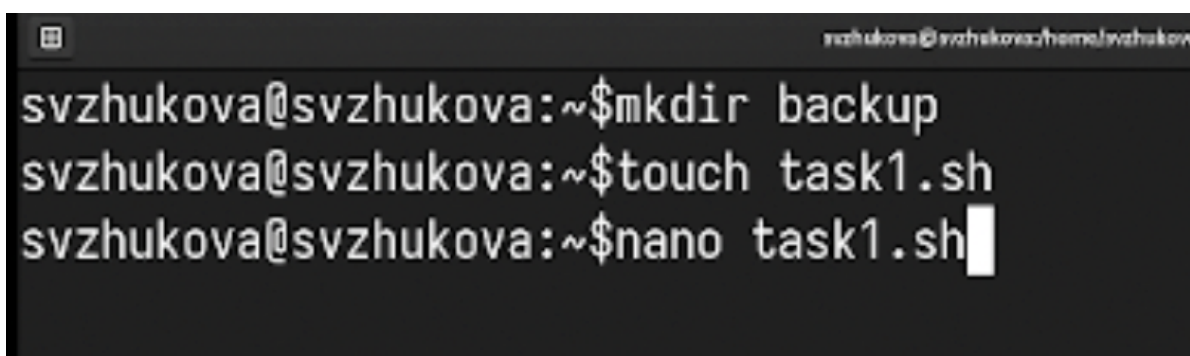
Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

## 2 Задание

1. Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в вашем домашнем каталоге. При этом файл должен архивироваться одним из архиваторов на выбор zip, bzip2 или tar. Способ использования команд архивации необходимо узнать, изучив справку.
2. Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Например, скрипт может последовательно распечатывать значения всех переданных аргументов.
3. Написать командный файл — аналог команды ls (без использования самой этой команды и команды dir). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога.
4. Написать командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки.

### 3 Выполнение лабораторной работы

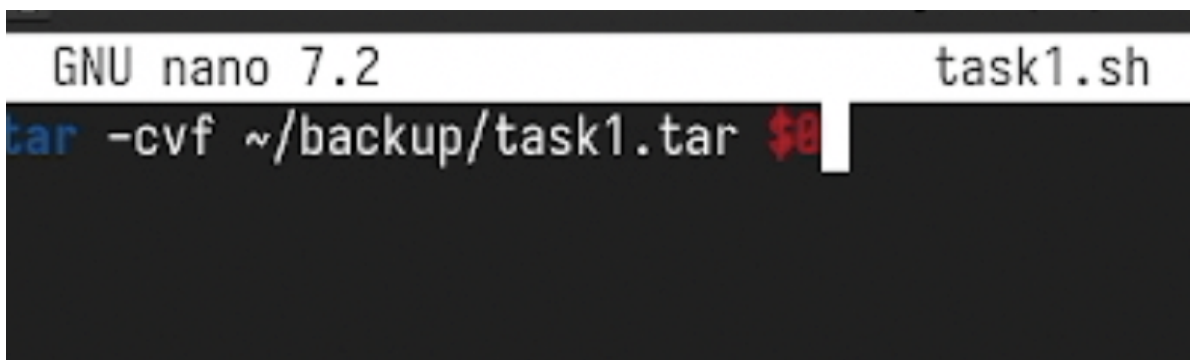
Откроем терминал и создадим в домашнем каталоге папку backup. После чего создадим файл lab1.sh для написания скрипта. (рис. 3.1).

A terminal window with a dark background. The prompt is 'svzhukova@svzhukova:~'. The user enters 'mkdir backup', then 'touch task1.sh', and finally 'nano task1.sh'. The cursor is at the end of the last command.

```
svzhukova@svzhukova:~$mkdir backup
svzhukova@svzhukova:~$touch task1.sh
svzhukova@svzhukova:~$nano task1.sh
```

Рис. 3.1: Создание нового каталога и файла для скрипта

В откроем созданный файл lab1.sh и приступим к написанию скрипта, который при запуске будет делать резервную копию самого себя в другую директорию backup в домашнем каталоге. При этом файл должен архивироваться tar. (рис. 3.2).

A screenshot of the nano text editor. The top bar shows 'GNU nano 7.2' on the left and 'task1.sh' on the right. The main editing area shows the command 'tar -cvf ~/backup/task1.tar \$0' followed by a red cursor. The background is dark.

```
GNU nano 7.2 task1.sh
tar -cvf ~/backup/task1.tar $0
```

Рис. 3.2: Написание первого скрипта

После того как скрипт написан мы сохраняем файл. В терминале мы даём этому файлу право на выполнение. Теперь запустим этот файл и перейдём в каталог backup для проверки командой ls (рис. 3.3).

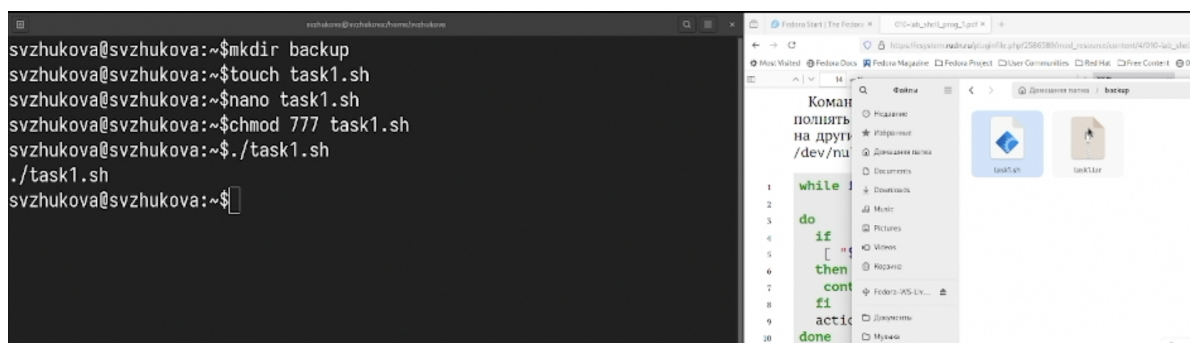


Рис. 3.3: Право на выполнение, запуск файла и проверка

Возвращаемся в домашний каталог и создаём второй файл для скрипта lab2.sh. (рис. 3.4).

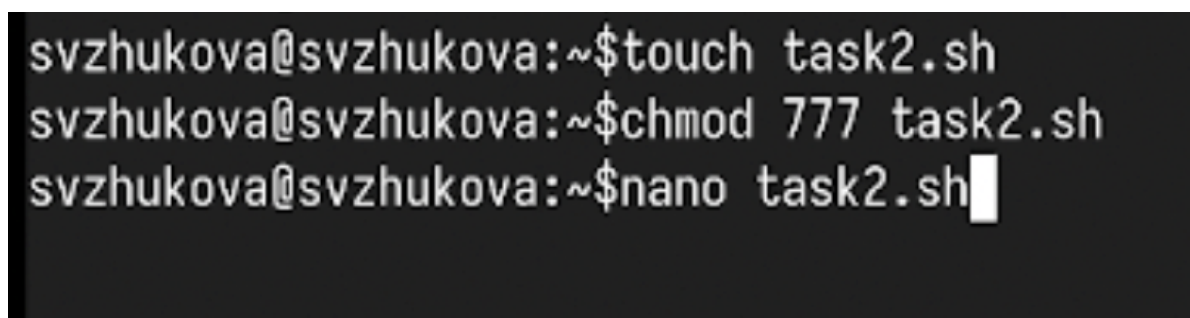
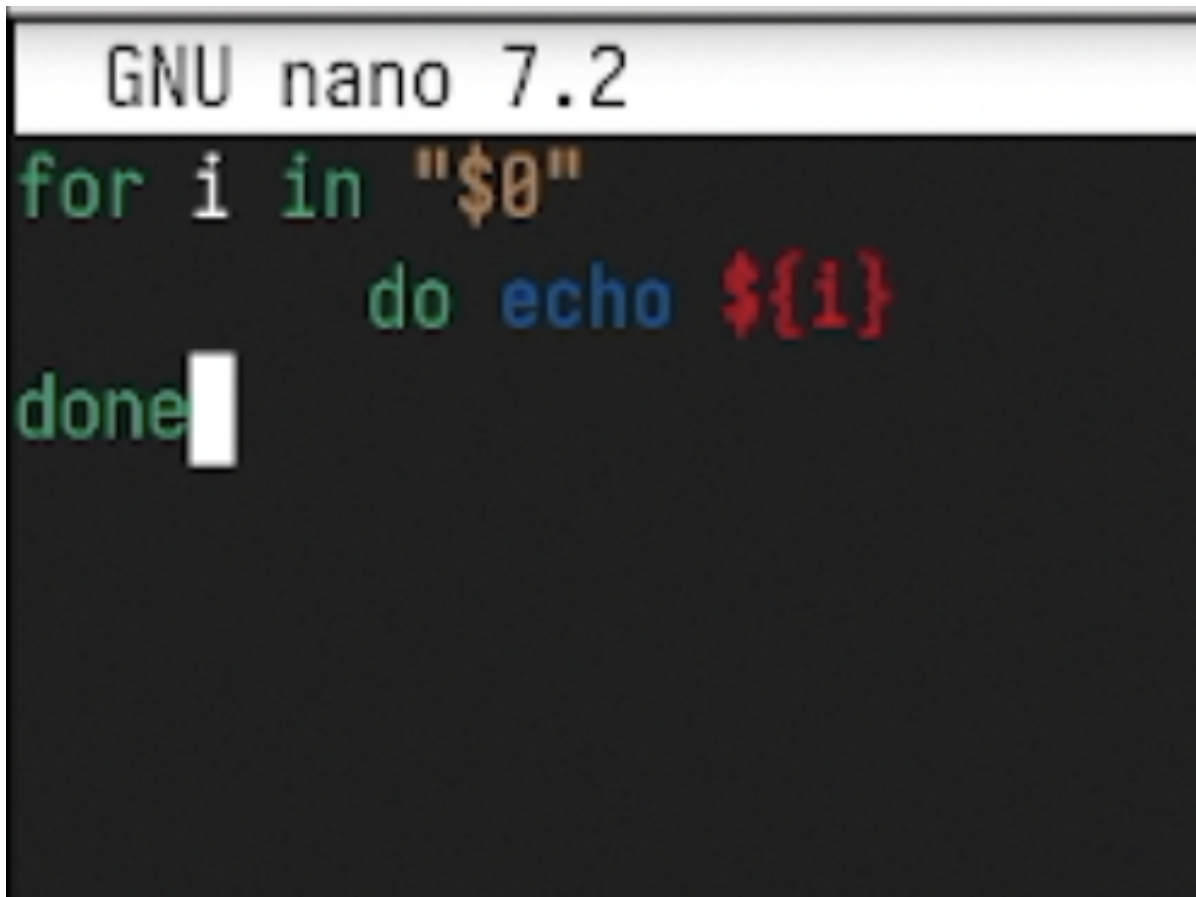


Рис. 3.4: Создание второго файла и открытие

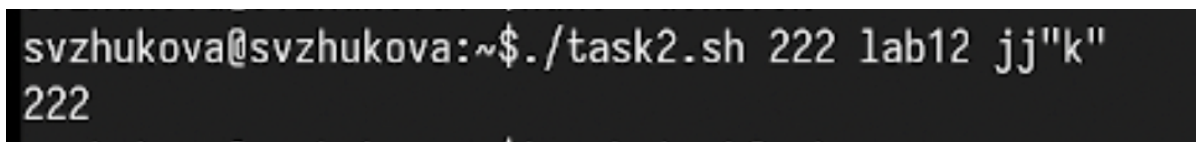
Открываем файл lab2.sh и начинаем писать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Скрипт может последовательно распечатывать значения всех переданных аргументов.(рис. 3.5).



```
GNU nano 7.2
for i in "$@"
do echo ${i}
done
```

Рис. 3.5: Написание второго скрипта

Сохраняем файл и также даём в терминале право на выполнение. Запускаем файл lab2.sh (рис. 3.6).



```
svzhukova@svzhukova:~$ ./task2.sh 222 lab12 jj"k"
222
```

Рис. 3.6: Право на выполнение, запуск файла

Снова переходим в домашний каталог и создаём третий файл. (рис. 3.7).

```
svzhukova@svzhukova:~$touch task3.sh
svzhukova@svzhukova:~$chmod 777 task3.sh
svzhukova@svzhukova:~$nano task3.sh
```

Рис. 3.7: Создание третьего файла

После открытия файла lab3.sh напишем командный файл — аналог команды ls (без использования самой этой команды и команды dir). В котором требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога. (рис. 3.8).

```
GNU nano 7.2 task3.sh
echo "$1/" | tr -d "\n";
stat --printf "%A" "$1/";
echo
for i in $1/*
do echo "${i} " | tr -d "\n";
stat --printf "%A" "${i}";
echo
done
```

Рис. 3.8: Написание третьего скрипта

Сохраняем наш скрипт и даём право на выполнение. Запускаем файл для каталога backup (рис. 3.9).

```
svzhukova@svzhukova:~$ ./task3.sh ~
/home/svzhukova/drwx-----
/home/svzhukova/123 -rw-r--r--
/home/svzhukova/22 -rw-r--r--
/home/svzhukova/2file.txt drwxr-xr-x
/home/svzhukova/32.png -rw-r--r--
/home/svzhukova/33 -rw-r--r--
/home/svzhukova/95-system-keyboard-config.conf -rw-r--r--
/home/svzhukova/abc1 -rw-rw-r--
/home/svzhukova/australia drwxr--r--
/home/svzhukova/backup drwxr-xr-x
/home/svzhukova/bin drwxr-xr-x
/home/svzhukova/conf.txt -rw-r--r--
/home/svzhukova/Documents drwxr-xr-x
/home/svzhukova/Downloads drwxr-xr-x
/home/svzhukova/DOWNLOADS -rw-r--r--
/home/svzhukova/exec -rw-r--r--
/home/svzhukova/feathers -rw-rw-r--
/home/svzhukova/file.txt -rw-r--r--
/home/svzhukova/git-extended drwxr-xr-x
/home/svzhukova/git@github.com:svzhukova drwxr-xr-x
/home/svzhukova/lab07.sh -rw-r--r--
/home/svzhukova/lab07.sh~ -rw-r--r--
/home/svzhukova/lab11 -rw-r--r--
/home/svzhukova/LICENSE -rw-r--r--
/home/svzhukova/may -rw-r--r--
/home/svzhukova/may~ -rw-r--r--
/home/svzhukova/monthly drwx--x--x
/home/svzhukova/my_os -r-xr--r--
```

Рис. 3.9: Право на выполнение, запуск файла для каталога backup

Переходим в домашний каталог и создаём последний файл для четвёртого скрипта. (рис. 3.10).

```
svzhukova@svzhukova:~$touch task4.sh
svzhukova@svzhukova:~$chmod 777 task4.sh
svzhukova@svzhukova:~$nano task4.sh
```

Рис. 3.10: Создание четвёртого файла

В четвёртом файле напишем командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки. (рис. 3.11).

```
GNU nano 7.2
let COUNT=0
for i in $2/*. $1
do let COUNT++
done
echo $COUNT
```

Рис. 3.11: Написание четвёртого скрипта

Сохраним файл. Как делали ранее, дадим файлу право на выполнение и запустим его для формата .txt (рис. 3.12).

```
svzhukova@svzhukova:~$ ./task4.sh txt ~
4
svzhukova@svzhukova:~$
```

Рис. 3.12: Право на выполнение, запуск файла для .txt

## 4 Выводы

Мы изучили основы программирования в оболочке ОС UNIX/Linux и научились писать небольшие командные файлы.

**Контрольные вопросы** 1. Понятие командной оболочки

**Командная оболочка** — это программа, которая предоставляет интерфейс для взаимодействия пользователя с операционной системой через командную строку. Она позволяет выполнять команды, управлять файлами, запускать программы и выполнять скрипты.

**Примеры командных оболочек:** - **Bash** (Bourne Again SHell) - **Zsh** (Z Shell) - **Csh** (C Shell) - **Tcsh** (an enhanced version of C Shell) - **Ksh** (Korn Shell)

**Основные отличия:** - **Bash** — наиболее распространенная оболочка. Поддерживает массивы и функции. - **Zsh** — расширенные функции интерактивности, работа с плагинами и тема оформления. - **Csh** — синтаксис, схожий с языком C; менее популярна, но имеет свои особенности, такие как управление историями. - **Ksh** — комбинация возможностей Bash и Csh.

---

### 2. Что такое POSIX?

**POSIX** (Portable Operating System Interface) — это стандарт, определяющий интерфейсы и утилиты для операционных систем на основе UNIX. Этот стандарт обеспечивает переносимость приложений между различными системами, что позволяет разработать программное обеспечение, которое будет работать на любой POSIX-совместимой системе.

### 3. Переменные и массивы в Bash

**Переменные** в Bash определяются следующим образом:

переменная=значение

Для доступа к значению переменной используется знак \$:

```
echo $переменная
```

**Массивы** определяются следующим образом:

массив=(значение1 значение2 значение3)

Доступ к элементам массива осуществляется с помощью индекса:

```
echo ${массив[0]}
```

### 4. Операторы let и read

**let** — это оператор для выполнения арифметических операций:

```
let "переменная = 5 + 3"
```

**read** — используется для ввода данных от пользователя:

```
read переменная
```

### 5. Арифметические операции в Bash

В Bash можно использовать следующие арифметические операции: - Сложение: + - Вычитание: - - Умножение: \* - Деление: / - Остаток от деления: %

Эти операции могут быть использованы с помощью команд `let`, `expr` или в арифметических выражениях.

### 6. Операция (( ))

Операция `(( ))` используется для арифметических выражений и позволяет выполнять расчеты без необходимости использования дополнительных команд, например:

```
(( переменная = 5 + 3 ))
```

При этом результат операции будет автоматически присвоен переменной.

## 7. Стандартные имена переменных

Некоторые стандартные имена переменных в Bash: - `$HOME` — домашний каталог пользователя. - `$USER` — имя текущего пользователя. - `$PATH` — список директорий, где ищутся исполняемые файлы. - `$PWD` — текущий рабочий каталог. - `$SHELL` — интерпретатор командной строки, который используется.

## 8. Что такое метасимволы?

**Метасимволы** — это специальные символы, используемые в оболочке для управления вводом и выводом, а также для работы с файлами и каталогами. К ним относятся: - `*` — соответствует любому количеству символов. - `?` — соответствует одному любому символу. - `[]` — используется для задания диапазона символов.

## 9. Экранирование метасимволов

Чтобы экранировать метасимволы, используется обратная косая черта (`\`). Например:

```
echo "Это символы: \* и \?"
```

Также можно использовать одинарные или двойные кавычки для предотвращения интерпретации метасимволов.

## 10. Создание и запуск командных файлов

### 11. Создать файл с расширением `.sh`:

```
nano myscript.sh
```

12. В начале файла указать интерпретатор:

```
#!/bin/bash
```

13. Сделать файл исполняемым:

```
chmod +x myscript.sh
```

14. Запустить файл:

```
./myscript.sh
```

15. Определение функций в Bash

Функции определяются следующим образом:

```
имя_функции() {  
    команда1  
    команда2  
}
```

Для вызова функции используется просто ее имя:

```
имя_функции
```

12. Как выяснить, является файл каталогом или обычным файлом?

Используйте команду `test` или встроенный `[:`

```
if [ -d имя_файла ]; then  
    echo "Это каталог"  
elif [ -f имя_файла ]; then  
    echo "Это обычный файл"  
fi
```

### 13. Назначение команд `set`, `typeset` и `unset`

- **set** — используется для установки или сброса параметров оболочки и переменных.
- **typeset** — позволяет объявлять локальные переменные в функциях, а также задавать тип переменной (например, массив).
- **unset** — используется для удаления переменной или функции.

### 14. Передача параметров в командные файлы

Параметры передаются в командные файлы при их вызове:

```
myscript.sh параметр1 параметр2
```

Внутри скрипта доступ к параметрам осуществляется через `$1`, `$2`, и так далее. `$@` и `$*` используются для обращения ко всем параметрам.

### 15. Специальные переменные языка Bash

Некоторые специальные переменные Bash: -  **\$?**  — код возврата последней выполненной команды. -  **\$#**  — количество переданных параметров. -  **\$\***  и  **\$@**  — все переданные параметры: `$@` сохраняет отдельные параметры, а `$*` объединяет их в одну строку. -  **\$\$**  — процесс ID текущего скрипта.